Build Plan: Architecture, Folders and Team Split

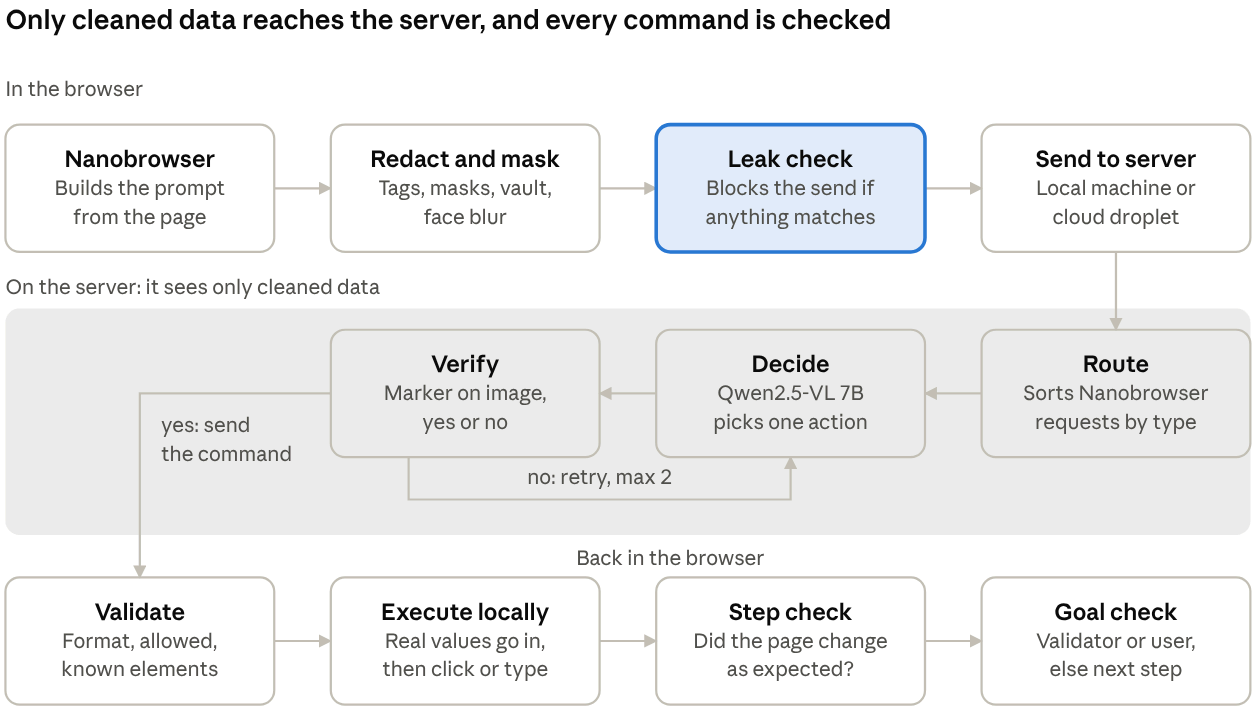
Oct 17, 2026 · @jayant

Scope and fixed choices

This version builds the privacy-preserving browser agent as a fork of Nanobrowser that talks to our own server. The server makes the decisions, so the extension only filters, sends and executes.

- **Shell:** Nanobrowser fork, Chrome only. All new client code lives in one `privacy/` folder so the changes are easy to review.
- **Model:** Qwen2.5-VL-7B-Instruct (Q4_K_M GGUF plus its vision projector file), served by `llama-server` on port 8000. One VLM does everything first. Splitting it into a vision model and a separate text decision model comes later, and only if tests show it helps.
- **Server:** our own program with an OpenAI-compatible endpoint, so Nanobrowser can use it as a custom provider.
- **Data:** fake personal details only, plus a throwaway Google account for any signed-in pages.
- **Open point:** a 7B vision model on an 8 GB GPU is tight. Fallbacks are listed under risks.

Flow



request to action · 11 steps, 1 retry loop

The leak check gates everything that leaves the browser. The server verifies point-based or risky actions with a marker before replying; the extension then validates the command, swaps real values back in, acts, and checks the page. If the goal is not met, the next step starts again.

The contract between client and server

Both sides code against two shapes, kept in `contracts/`, so the client and the server can be built in parallel.

Request (client to server). The standard OpenAI-style chat request that Nanobrowser already sends, after redaction: the text carries typed tags such as `EMAIL_1`, and the screenshot is the masked, shrunk image. One extra header, `X-Redaction-Scheme: typed-tags-v1`, tells the server how the data was redacted.

Command (server to client), returned as the reply content:

```
{ "action": "type", "element_index": 15, "point": null, "value_ref":
"NAME_1", "risk": "low", "reason": "Name field of the form" }
```

- Allowed actions: `click`, `type`, `scroll`, `ask_user`, `done`.
- `element_index` is used whenever the page element exists. `point` (x, y in screenshot pixels) is used only when it does not.
- `risk` is `high` for Submit, Pay, Delete and similar actions, which always need the user's confirmation.

The server's formatter turns this command into whatever shape Nanobrowser's own agent expects. The first spike, which logs what Nanobrowser sends and expects, shows what that shape is.

Folder structure

The client lives inside the Nanobrowser fork, and everything we add sits in `privacy/` folders. The paths inside the fork follow my understanding of its layout, so confirm them after cloning.

```
privacy-agent/
├─ README.md
├─ LICENSE
├─ contracts/                       shared shapes everyone codes against
|  ├─ request.schema.json
|  └─ command.schema.json
├─ client/                          fork of Nanobrowser (Chrome extension)
|  ├─ background/privacy/          NEW: all client-side privacy code
|  |  ├─ netWrapper.ts             intercepts model calls: redact on the way
out, swap values back on the way in
|  |  ├─ rules/
|  |  |  ├─ builtinRules.ts        layer 1: hardcoded patterns and field
rules
|  |  |  └─ learnedRules.ts        saved "hide this" patterns (local storage
only)
|  |  ├─ detectors/
|  |  |  ├─ textDetector.ts        applies the rules to text
|  |  |  ├─ similarity.ts          layer 2: small embedding match for similar
labels
|  |  |  └─ faceDetector.ts        layer 2: face detection (stretch)
|  |  ├─ redactor.ts               text to typed tags such as EMAIL_1
|  |  ├─ imageMasker.ts            draws masks on the screenshot
|  |  ├─ vault.ts                  tag to real value, kept local
|  |  ├─ leakCheck.ts              last scan of the outgoing payload
|  |  ├─ validator.ts              checks every server command before it runs
```

```
|  |  └─ memory/                    stretch: recipes and mistake notes
|  ├─ content/rectCollector.ts      NEW: screen rectangles of redacted
elements
|  └─ side-panel/
|     ├─ PreviewPanel.tsx           NEW: what the user sees vs. what the
server sees
|     └─ RulesEditor.tsx            NEW: view and undo learned rules
├─ server/
|  ├─ README.md                     how to start llama-server and this server
|  ├─ app/
|  |  ├─ main.py                    OpenAI-compatible endpoint
|  |  ├─ router.py                  tells Nanobrowser's request types apart
|  |  ├─ vlm_client.py              talks to llama-server on port 8000
|  |  ├─ pipeline/
|  |  |  ├─ decide.py               call 1: image + page list + request gives
one action
|  |  |  ├─ verify.py               call 2: marker on the image, yes or no
|  |  |  └─ retry.py                new input each retry, at most two
|  |  ├─ imaging.py                 draws markers and grids on the clean image
|  |  ├─ formatter.py               shapes the command for Nanobrowser
|  |  ├─ prompts/                   decide.txt and verify.txt
|  |  └─ request_log.py             saves every request and reply as demo
evidence
|  └─ tests/
├─ eval/
|  ├─ pages/                        fake-PII test pages, two layouts each
|  ├─ answer_keys/
|  ├─ run_benchmark.py
|  └─ results/
└─ docs/
```

The server is written in Python here, but any language works as long as it keeps the contract.

Team split

Two people carry the heavy parts, the client privacy layer and the server pipeline. The other two cover testing and the user-facing pieces. Names are still to be filled in.

Person	Load	Owns	Done when
Person 1	Heavy	Client privacy layer: rules, detectors, typed tags and vault, screenshot masking, leak check, network wrapper, command validator, swapping real values back in	A request leaves the browser with only tags and masked images, and the value typed into the page is the real one
Person 2	Heavy	Server pipeline: OpenAI-compatible endpoint, request router, VLM client, decide and verify prompts, marker drawing, retry policy, formatter	Nanobrowser finishes a task through our server, with one or two model calls per step in the common case
Person 3	Light	Testing and evaluation: fake-PII pages in two layouts, answer keys, baseline log from stock Nanobrowser, benchmark script, results tables	A table comparing stock Nanobrowser and ours: success, time, personal strings found in requests
Person 4	Light	Side panel and project wrapper: preview panel (sent vs. seen), "hide this" button and rules editor, README, license, demo script	A user can see and correct what is sent, and the repo is ready to publish

How the work connects:

- Persons 1 and 2 meet at the contract in `contracts/`. Agree it before anyone writes code.
- Person 3 captures the stock Nanobrowser baseline in the first hour by using the logging proxy.
- Person 4 builds the panel on mock data until Person 1's redaction output is ready.
- Stretch goals go to whoever finishes first: face blur, then recipe memory.

Build order and checkpoints

Build the privacy core first, so something demoable exists early. Each phase ends with a checkpoint the whole team can check.

1. **Contract and spike (everyone).** Agree `contracts/`, clone the fork, start `llama-server` with the model, and run a small logging proxy as Nanobrowser's custom provider. *Checkpoint:* we know what stock requests and replies look like, and the stock leak baseline is captured.
2. **Core path.** Person 2 returns commands for one form, Person 1 redacts text and swaps tags back, Person 3 finishes the test pages, Person 4 builds the panel on mock data. *Checkpoint:* the Google Form is filled end to end with only tags leaving the browser.
3. **Safety and checks.** Screenshot masks, leak check, command validator, confirmation for risky actions, and the marker verify step. *Checkpoint:* the request log shows no real values, and wrong clicks are caught.
4. **Learning and extras.** Learned rules from the panel's "hide this" button, then face blur, then recipe memory.
5. **Benchmark and rehearsal.** Run stock Nanobrowser against ours on three tasks, fill the tables, and rehearse one task until it works every time.

Cut first if time runs short: recipe memory, face blur, and the two-model split.

Risks and fallbacks

- **The 7B model may not fit comfortably in 8 GB.** The weights, the vision projector (offloaded to the GPU by default) and the image tokens all compete for memory. Use small screenshots and a short context. Fallbacks: Qwen2.5-VL 3B, or Qwen3-VL 4B or 8B. Because the endpoint is OpenAI-compatible, switching is one setting.
- **Several calls per step.** Nanobrowser uses a Planner, a Navigator and a Validator. Run our pipeline only on the request that chooses the action and pass the others straight through. If that gets messy, move to our own command channel.
- **The VLM may break the expected format.** Use strict JSON prompts, temperature 0, schema validation, and a capped retry.
- **The network wrapper may miss model calls** if Nanobrowser's model library captures the browser's `fetch` early. Install the wrapper first in the background script, or patch the provider code.
- **Overclaiming.** Say "only cleaned data crosses to the server", publish the detection recall number, and state the limits of the redacted screenshot openly.

Open questions:

- [ ] Hours available on the day
- [ ] Names for Persons 1 to 4

- [ ] Results of the first spike: what Nanobrowser sends, and seconds per step with the 7B model